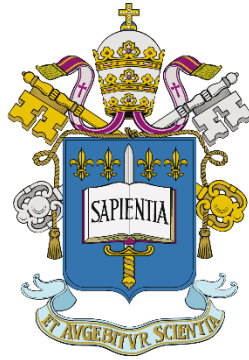


Pontifícia Universidade Católica de São Paulo
PUC-SP



PUC-SP

Vinicius de Lucena

Classificador EMNIST Balanced: Classificador Alfanumérico com MLP (Multi-Layer Perceptron) no Framework PyTorch

Ciência de Dados e Inteligência Artificial

São Paulo
2026

SUMÁRIO

1	INTRODUÇÃO	1
2	FUNDAMENTAÇÃO TEÓRICA	2
2.1	REDE NEURAL MLP	2
3	METODOLOGIA	3
3.1	DATASET E PIPELINE DE DADOS	3
3.2	CORREÇÃO DA ORIENTAÇÃO	3
3.3	PIPELINE DE DADOS.....	3
3.4	ARQUITETURA DOS MODELOS	4
3.5	ESTRATÉGIA DE TREINAMENTO	4
4	RESULTADOS E DISCUSSÃO	5
4.1	COMPARAÇÃO DOS MODELOS.....	5
4.2	ANÁLISE DE ERROS.....	6
4.3	ENSEMBLE E TTA (TEST-TIME AUGMENTATION).....	6
5	APLICAÇÃO NO STREAMLIT	7
6	CONCLUSÕES	8
	REFERÊNCIAS.....	9

1 INTRODUÇÃO

O trabalho desenvolvido aborda o problema da classificação de caracteres manuscritos alfanuméricos utilizando principalmente a arquitetura MLP (Multi-Layer Perceptron), modelo esse que foi abordado e testado em sala de aula.

O objetivo é implementar o pipeline completo de deep learning (arquitetura, treinamento, persistência e deploy em Streamlit) e obter um resultado de classificação satisfatório no dataset escolhido.

O dataset escolhido foi o EMNIST Balanced, uma variação do MNIST que adiciona letras maiúsculas e minúsculas. Entre as variantes disponíveis (byclass, bymerge, letters, mnist), a balanced foi escolhida por oferecer 47 classes com o mesmo número de exemplos, evitando desbalanceamento, e por representar um desafio maior do que o MNIST tradicional, sendo uma escolha pessoal.

2 FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta os conceitos teóricos que fundamentam a arquitetura do trabalho.

2.1 REDE NEURAL MLP

Uma rede MLP (Multi-Layer Perceptron) é composta por camadas em que cada neurônio de uma camada se conecta a todos os neurônios da camada seguinte, a tornando uma rede totalmente conectada.

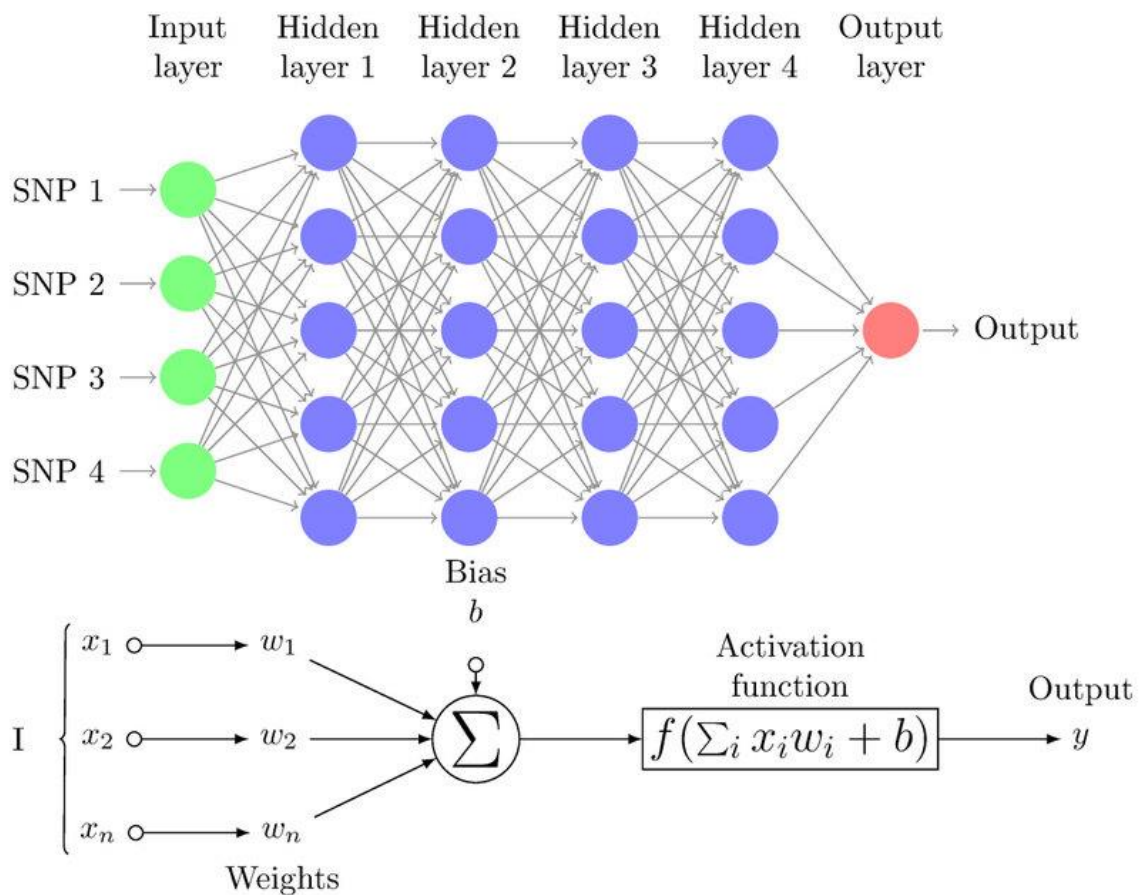


Figura 1 - Estrutura MLP (Imperial GitHub Documentation)

Em nosso contexto, ela deve operar sobre imagens 28x28, e como o modelo é extremamente sensível a variações devido a sua estrutura, é necessário achatar a imagem (flatten) e garantir que todas tenham a mesma dimensão, esse achatamento transforma a matriz bidimensional em um vetor de 784 posições. O achatamento descarta a informação espacial, fazendo com que a rede aprenda apenas as relações entre os pixels, enquanto modelos mais sofisticados, como as redes convolucionais, conseguem resolver esse problema ao aplicar filtros locais.

3 METODOLOGIA

Descrição do Dataset, Pipeline de dados, arquitetura e treinamento.

3.1 DATASET E PIPELINE DE DADOS

O EMNIST Balanced contém 112.800 exemplos de treino e 18.800 de teste, distribuídos de forma balanceada entre as 47 classes, sendo 10 dígitos de 0 a 9 e 37 classes de, sendo 26 maiúsculas e 11 minúsculas. As letras que têm forma maiúscula idêntica ou extremamente semelhante à sua versão minúscula tiveram sua classe unificada (C com c, e X com x, por exemplo).

3.2 CORREÇÃO DA ORIENTAÇÃO

Os arquivos do EMNIST disponibilizados pelo NIST e indexados pelo torchvision apresentam orientação espelhada em relação ao MNIST tradicional, sem treinamento, as imagens aparecem deitadas, para efeito de treinamento e visualização, reorganizei a posição das imagens do dataset.



Figura 2 - Correção da Orientação (Notebook .ipynb)

3.3 PIPELINE DE DADOS

As imagens foram carregadas via torchvision.datasets.EMNIST e processadas por uma pipeline de transforms que inclui a correção de orientação, a conversão para tensor (ToTensor) e a normalização (Normalize) com média e desvio-padrão do dataset. Os batches foram fornecidos ao modelo via DataLoader do PyTorch, com shuffle = True no conjunto de treino.

O conjunto de treino original foi separado em treino e validação num split 90/10, com semente (seed) fixa (42) para garantir reprodutibilidade, totalizando 101.520 exemplos de treino e 11.280 de validação. O conjunto de teste foi isolado e utilizado apenas ao final, após a seleção do checkpoint, a fim de evitar vazamento de informação.

3.4 ARQUITETURA DOS MODELOS

No desenvolvimento do projeto, decidi criar dois modelos MLP e compará-los, os nomeei MLP_A e MLP_B, sua diferença estrutural é simples, uma arquitetura padrão em ambos, porém o MLP_B teve a adição da função BatchNorm1d após cada camada linear, essa diferença sutil vem com a intenção de testar ambos os modelos, e comparar na prática, o impacto da normalização por batch após cada etapa. Em todas as camadas ocultas eu apliquei o ReLU, pois além de não saturar os valores positivos, evita problemas de desaparecimento do gradiente, problema presente em ativações como sigmoide e tanh. Também foi utilizado Dropout gradual, com taxa maior (0,3) após a primeira camada oculta e menor (0,2) após a segunda, como estratégia de regularização para reduzir overfitting.

3.5 ESTRATÉGIA DE TREINAMENTO

O treinamento de ambas as arquiteturas foi conduzido sob hiperparâmetros idênticos, para garantir uma comparação apenas dada pela diferença de arquitetura.

- Função de Perda: CrossEntropyLoss, a função de perda padrão para classificações multiclasse, medindo o quão longe a distribuição de probabilidades prevista está da classe verdadeira
- Otimizador: Adam, learning rate de 0,001 e weight decay de 0,0005.
- Scheduler: Utilizei o StepLR com step_size = 8 e gamma = 0.5, ou seja, a cada 8 iterações (schedule), o learning rate diminui.
- Paciência: Para as iterações, a fim de evitar treinamento desnecessário, utilizei uma tolerância de 5 iterações sem ganho de performance/perda de loss.
- Checkpoint: Depois de 40 épocas (Número decidido com base em tempo (média de treinamento superior a 1 hora), já que o modelo foi rodado localmente) ou em caso de Early Stop (Paciência), a melhor época é salva localmente em formato “.pth”, garantindo maior eficácia e aproveitamento do treinamento, evitando pegar a última época, que nem sempre é a melhor.

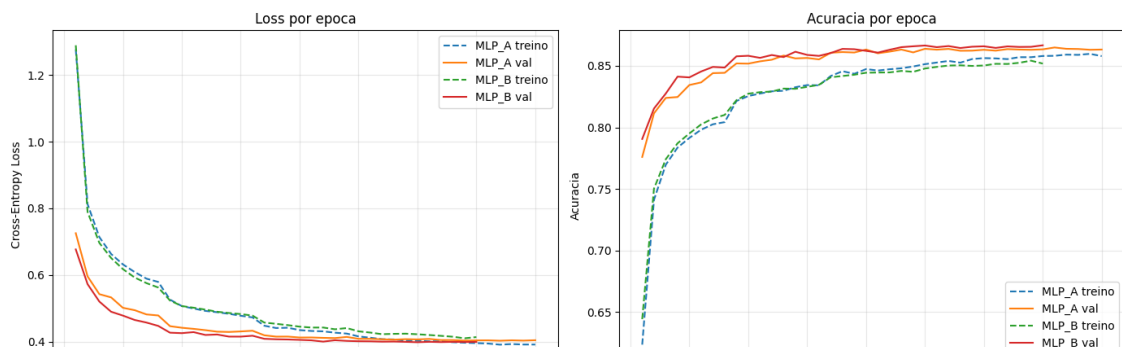


Figura 3 - CrossEntropyLoss e Acurácia dos Modelos (Notebook .ipynb)

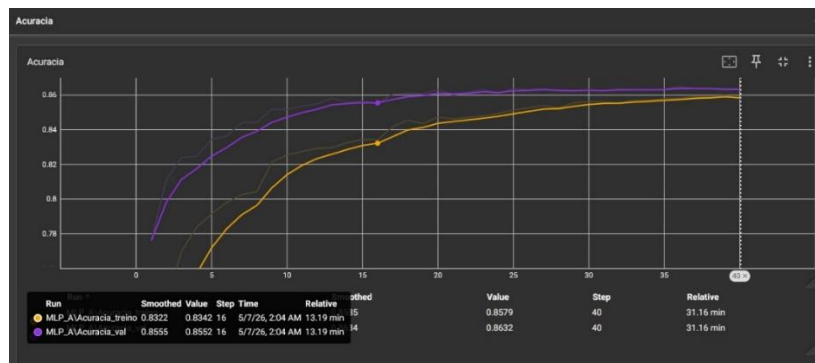


Figura 4 - Acurácia do Modelo MLP_A (Tensorboard)

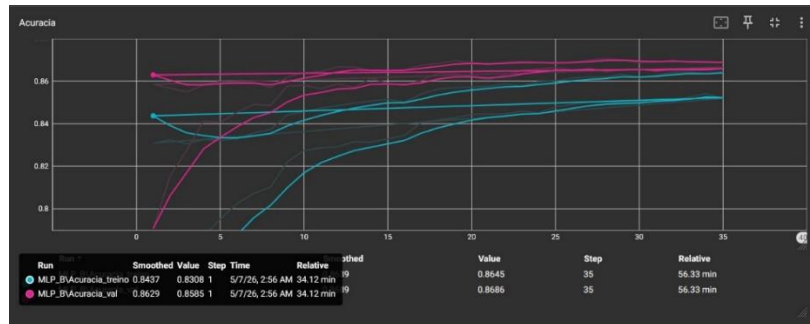
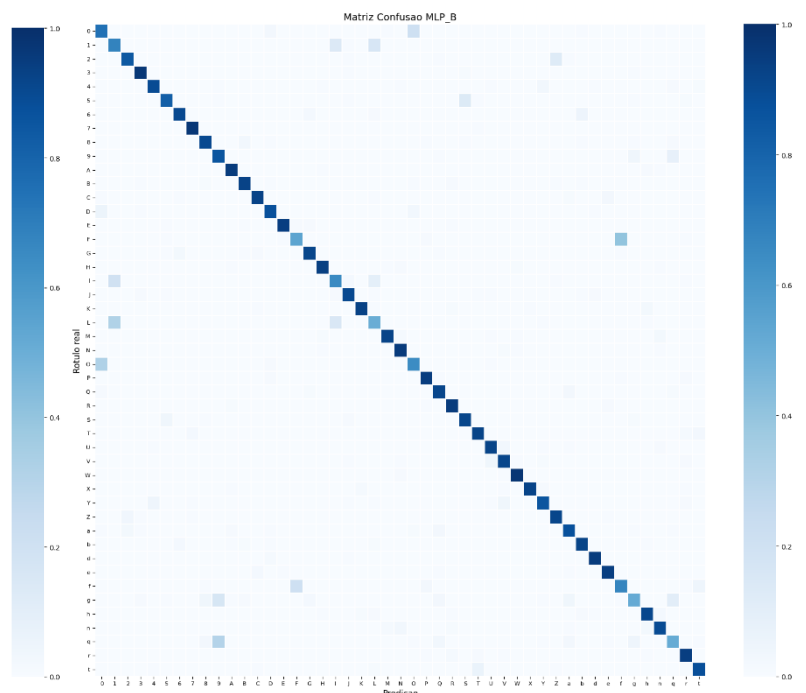
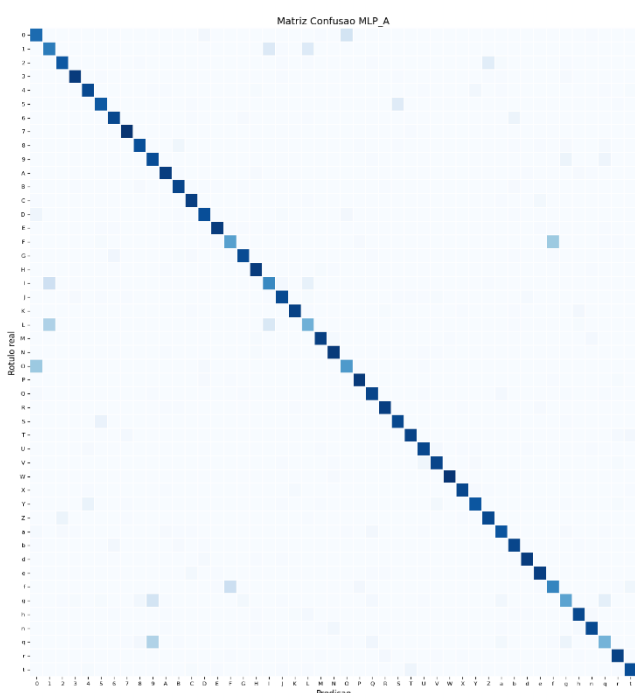


Figura 5 - Acurácia do Modelo MLP_B (Tensorboard)

4 RESULTADOS E DISCUSSÃO

4.1 COMPARAÇÃO DOS MODELOS

Após o treinamento, realizei uma avaliação no conjunto de teste, para assim ver a diferença na prática da estrutura dos modelos, em uma das avaliações (Valores variam de acordo com runs de treinamento) obtive 85,87% de acurácia para o modelo MLP_A e 86,20% para o MLP_B, resultando de uma diferença de 0,33 pontos percentuais, o que tende a mostrar a importância do Batch Normalization.



Figuras 6 e 7 – Matrizes de Confusão MLP_A e MLP_B (Notebook .ipynb)

4.2 ANÁLISE DE ERROS

Além da acurácia global, a ferramenta `classification_report` do `scikit-learn` reporta precisão, recall e F1 por classe, e a matriz de confusão demonstra a relação de acertos/erros, e visualmente conseguimos destacar quais as situações em que o modelo tem facilidade ou dificuldade de acertar, quanto às dificuldades, grande parte delas vem da ambiguidade do formato do caractere, podemos citar a semelhança entre:

- I e l (I e L)
- O e 0 (O e Zero)
- B e 8, entre outros



Figura 8 - Predições (Notebook .ipynb)

4.3 ENSEMBLE E TTA (TEST-TIME AUGMENTATION)

Para aprimorar os resultados sem necessidade de retreinamento, implementei duas técnicas de inferência: Ensemble e TTA (Test-Time Augmentation).

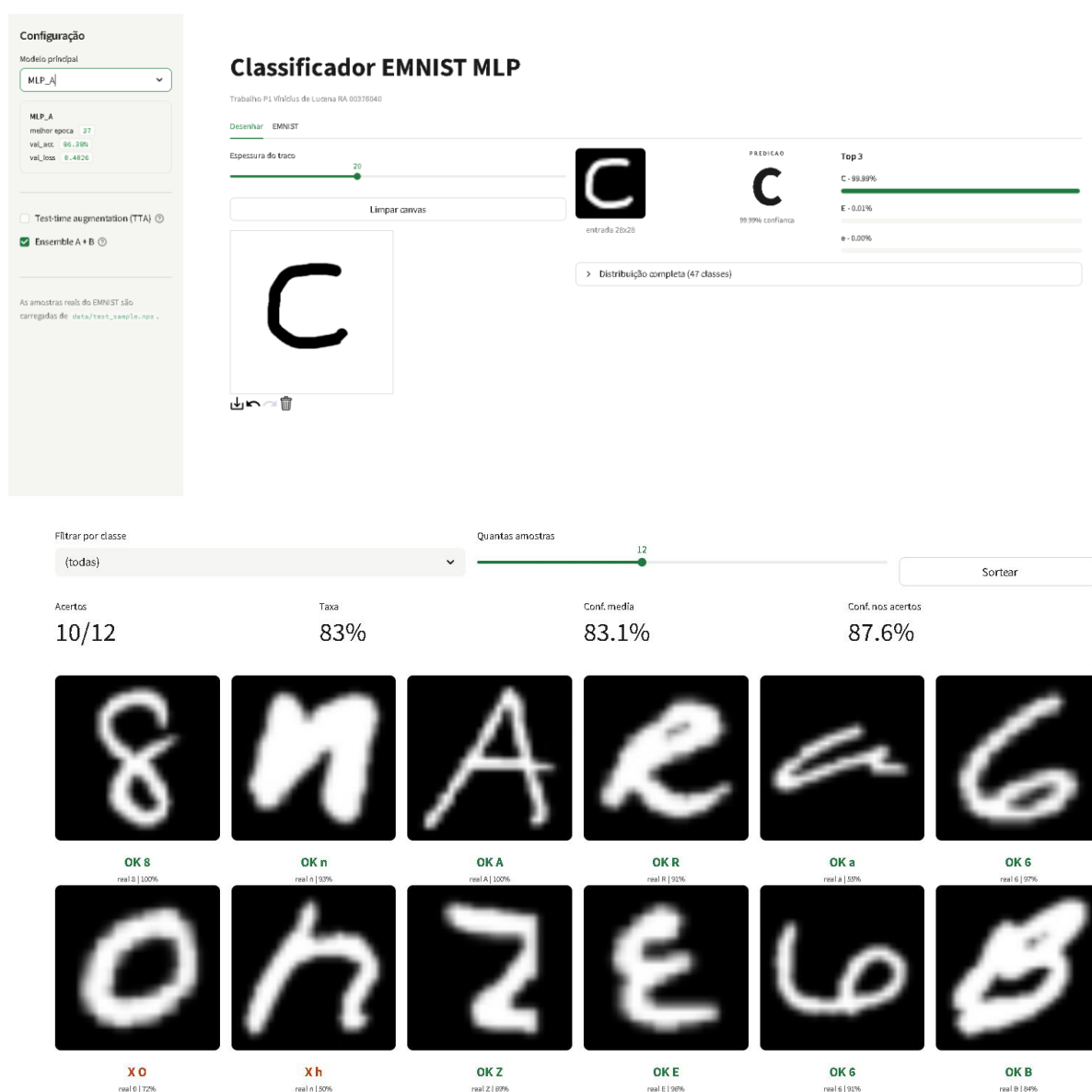
Ensemble: média das probabilidades (softmax) das saídas dos dois modelos. O softmax normaliza as saídas, garantindo peso equivalente entre os modelos na classificação final.

TTA: além da inferência padrão, desloca a imagem em 1 pixel em cada uma das 4 direções e realiza inferências adicionais, retornando a média das probabilidades das 5 predições.

As duas técnicas também podem ser combinadas, totalizando 10 inferências por amostra. Na prática, os ganhos medidos foram de 0,47 p.p. (Ensemble), 0,16 p.p. (TTA) e 0,55 p.p. (Ensemble + TTA) em relação à melhor acurácia individual.

5 APLICAÇÃO NO STREAMLIT

Para melhor visibilidade e efeitos de teste, desenvolvi com o auxílio de IA Generativa uma aplicação com interface visual que testa os modelos e ferramentas por meio de desenhos, permitindo o usuário fazer seu próprio caractere e testar a classificação do modelo, a aplicação está disponível em emnistmlp.streamlit.app, e é alimentada diretamente pelo [meu repositório GitHub](#). A aplicação mostra também inferências reais do dataset de testes, possibilitando visualização fácil da acurácia do modelo, elencando as 3 maiores probabilidades (com opção de mostrar todas).



Figuras 9 e 10 – Aplicação Streamlit

6 CONCLUSÕES

O projeto, guiado por materiais de aula e referências externas, resultou em uma solução com acurácia adequada para o problema proposto. A comparação entre duas arquiteturas (com e sem BatchNorm), somada às técnicas de Ensemble e TTA, contribuíram para um entendimento mais concreto do papel de cada componente de uma rede neural e do impacto de escolhas de inferência.

Apesar de o MLP não ser a melhor ferramenta no contexto de imagens (por descartar a informação espacial e ser sensível a perturbações), o domínio de sua estrutura e funcionamento foi suficiente para desenvolver uma solução viável e adequada para o problema.

As ferramentas de IA generativa foram utilizadas como apoio na construção da aplicação Streamlit e na revisão pontual de código e texto. A arquitetura dos modelos, a análise dos resultados e as decisões técnicas foram de minha autoria, com base principalmente nos conteúdos abordados em sala de aula.

REFERÊNCIAS

Multilayer Perceptron Classification - ReCoDE- AI For Patents. Disponível em: <https://imperialcollegelondon.github.io/ReCoDE-AIForPatents/2_Multilayer_Perceptron_Classification/>.

EMNIST — Torchvision main documentation. Disponível em: <<https://docs.pytorch.org/vision/main/generated/torchvision.datasets.EMNIST.html>>. Acesso em: 29 abril. 2026.

The EMNIST dataset. Disponível em: <<https://www.nist.gov/itl/products-and-services/emnist-dataset>>. Acesso em: 29 abril. 2026.

PyTorch documentation — PyTorch 2.11 documentation. Disponível em: <<https://docs.pytorch.org/docs/2.11/index.html>>. Acesso em: 1 maio. 2026.